An Empirical Evaluation of Source Code Chunking and Context Headers for Requirement Evidence Retrieval

Anonymous Authors

***Abstract*—Locating source code that evidences a technical requirement is a common need in software analysis. Syntax-aware chunking is motivated by its preservation of code structure, yet chunk boundaries are confounded with the context metadata prepended to each chunk. In this Research Preview, we cross two chunking strategies, abstract syntax tree (AST)-first chunking with line fallback and 50-line windows, with and without a hierarchical context header. We evaluate them on 25 job descriptions, whose required skills serve as requirements, and 40 public repositories (872 chunks), using BM25, dense retrieval, and cross-encoder reranking against 882 human-labeled pairs. After reranking, headers improved NDCG@10 for AST chunks from 0.341 to 0.487 (Holm-adjusted $p = 0.0001$), and without headers, line windows outperformed AST chunks (0.580 vs. 0.341, $p_{\text{Holm}} < 0.001$). Line windows without headers significantly outperformed AST chunks with headers ($\Delta = -0.093$, $p_{\text{Holm}} = 0.0438$). Line windows with headers scored highest (0.621), significantly above AST with headers in an exploratory comparison ($\Delta = -0.134$, $p = 0.0003$). Following parser upgrade, AST-arm fallback rate was reduced to 0.0%, and 25 queries provide adequate power for primary differences. We outline findings on isolating header components and empirical implications for code retrieval.**

***Index Terms*—Source code retrieval, code chunking, abstract syntax trees, context headers, requirements traceability, empirical software engineering**

I. Introduction

Locating code that provides evidence for a written requirement is a recurring need in software analysis, for example when checking whether a repository supports the requirements of a task. Tracing requirements to code has a long history in software engineering [**?**], [**?**], [**?**]. We study a traceability-style variant: given a technical requirement phrased in natural language, here a skill required by a job description (JD), retrieve the code chunks that evidence it.

Chunking via abstract syntax trees (ASTs) is motivated by the observation that line-based splitting can break semantic structures [**?**]. Recent controlled evidence in code completion, however, finds that structure-aware chunking does not outperform a simple sliding window [**?**], and another study of code-RAG design choices reports that line-based chunking matches syntax-aware splitting across context budgets [**?**]. Whether this holds when the goal is to retrieve evidence for a requirement is unclear to us. Moreover, real-world source files can contain constructs, such as declarative configurations, UI templates, and type declarations, that standard method-level AST extraction does not cover, and chunk boundaries are often confounded with the metadata (e.g., file path, class name) prepended to chunks. To the best of our knowledge, no prior study crosses chunking strategy with context headers for requirement-to-code evidence retrieval with human relevance labels.

In this Research Preview, we investigate how chunk boundary formulation and contextual metadata interact to determine retrieval effectiveness. We formulate four research questions:

- **RQ1 (Preliminary):** Does prepending a hierarchical context header improve the retrieval of requirement evidence from source code chunks?
- **RQ2 (Preliminary):** Does AST-first chunking outperform line-window chunking when both conditions use, or both omit, context headers?
- **RQ3 (Planned):** Which context header components drive retrieval improvements, and how do chunking trade-offs generalize across programming paradigms?
- **RQ4 (Planned):** Does retrieved code evidence improve human evaluators' accuracy and confidence in downstream requirement verification?

To address RQ1 and RQ2, we run a controlled 2×2 factorial experiment on 25 public JDs and 40 open-source repositories (20 Java Spring Boot, 20 React/TypeScript; 798 chunks) at three retrieval tiers (BM25, dense BGE-M3, and cross-encoder reranking). Preliminarily, headers improve reranking of AST chunks ($\Delta = +0.112$, $p_{\text{Holm}} = 0.0031$); without headers, line windows outperform AST chunks ($\Delta = -0.126$, $p_{\text{Holm}} = 0.0001$); and we find no evidence that AST chunks with headers differ from line windows without headers ($\Delta = -0.014$, $p_{\text{Holm}} = 0.85$).

We make four contributions:

1) A factorial design that crosses chunking strategy with context headers for requirement-to-code evidence retrieval over multi-file repositories.
2) Preliminary findings: headers improve reranking of AST chunks, headerless line windows outperform AST chunks, and we find no evidence that AST chunks with headers differ from headerless line windows; an audit shows that 35.7% of AST-arm chunks are line-window fallbacks.
3) A replication package with 603 human-labeled JD-chunk pairs (weighted kappa 0.78 overall, 0.88 on the initial gold subset, and 0.68 on the pooled items) and retrieval runs.
4) A research plan to test why headers help and whether retrieved evidence supports human verification.

The study relates to SANER topics on software analysis, mining software repositories, and program comprehension.

II. Related Work

We organize related work into four areas: code chunking for retrieval-augmented generation, dense code retrieval with reranking, context enrichment, and requirements traceability with skill mining.

A. Code Chunking for Retrieval-Augmented Generation

Line-based chunking can split functions or merge unrelated code [**?**]. cAST [**?**] addresses this by recursively splitting large AST nodes and merging sibling nodes under size limits, and reports gains on RepoEval retrieval and SWE-bench generation. Later controlled evidence is less favorable to structure: Wu et al. [**?**] compared four chunking strategies (function, declaration, sliding window, and cAST) for retrieval-augmented code completion and found that structure-aware strategies did not outperform a sliding window, with function-level chunking performing worst; Galimzyanov et al. [**?**] likewise report that line-based chunking matches syntax-aware splitting across context budgets. These studies target code completion and related tasks. We study human-labeled requirement-to-code evidence retrieval and also cross chunking with context headers.

B. Dense Code Retrieval and Reranking

Neural code search maps natural-language queries and code into a shared vector space [**?**], [**?**], [**?**], and multilingual embedders such as BGE-M3 [**?**] encode queries and chunks independently. Because a bi-encoder compresses a chunk into one vector, pipelines often add a cross-encoder that re-ranks the top candidates [**?**], [**?**]: TOSS [**?**] combines IR-based and bi-encoder recall with cross-encoder reranking, and CoSTV [**?**] uses a two-stage design to reduce search time while retaining accuracy. We use the same recall-then-rerank structure and ask how chunk shape and headers interact with the reranker.

C. Context Enrichment

Isolated chunks can lack the surrounding class and package context. RepoCoder [**?**] retrieves repository context iteratively to improve completion, and RepoBench [**?**] and CrossCodeEval [**?**] benchmark repository-level completion that needs cross-file context. These works enrich the prompt with retrieved context. We ask a narrower question: does a short hierarchical header prepended to each chunk change retrieval, and does it do so equally for AST chunks and line windows?

D. Requirements Traceability and Skill Mining

Tracing natural-language requirements to code is a long-standing problem [**?**]; recent work addresses the abstraction gap between artifacts [**?**] and the evolution of trace links [**?**]. Separately, repository mining has been used to estimate developer expertise; for example, Montandon et al. [**?**] predicted expertise in libraries and frameworks from GitHub activity using machine-learning classifiers with survey-based ground truth. We take the traceability view: a required skill is treated as a requirement, and we ask which code chunks evidence it. We do not assess individual developers.

III. Approach and Study Design

To separate chunk boundaries from context metadata, we use a 2×2 factorial design evaluated at three retrieval tiers, with an analysis plan frozen before the evaluation runs.

A. Corpus and Query Curation

Our corpus comprises 40 open-source GitHub repositories (20 Java Spring Boot, 20 React/TypeScript), encompassing 189 source files. Repository licenses include 19 MIT (47.5%), 16 Public GitHub default copyright (40.0%), 4 Apache-2.0 (10.0%), and 1 GNU AGPL-3.0 (2.5%). Requirements are extracted from 25 public job descriptions (15 Java backend, 10 React/TypeScript frontend) with a frozen prompt run on `gemini-1.5-flash` (recorded in `queries_frozen.json`); each query lists the mandatory skills.

B. Factorial Conditions and Retrieval Pipeline

We evaluate two orthogonal factors:

1) **Chunking Strategy:** *Line-window chunking* slices source code into 50-LOC windows with a 10-LOC overlap (step 40 LOC), representing the standard language-agnostic baseline. *AST-first chunking with line fallback (AST arm)* uses Tree-sitter to parse source files into concrete syntax trees, extracting discrete functional boundaries (`method_declaration`, `constructor_declaration`, `arrow_function`, and function expressions). When a file has no extractable function (e.g., top-level JSX, styled-components, type declarations, or configuration objects), the parser falls back to 50-LOC windows so that such files remain searchable. In total, our corpus yields 434 line chunks and 364 AST-arm chunks (234 pure AST method extractions, 130 line-window fallback chunks).
2) **Context Header Condition:** In the *Without Header* condition, chunks contain solely the raw code text. In the *With Header* condition, chunks are prepended with a standardized hierarchical context prefix: `// File: <path> \n // Class: <name> \n // Method: <name> \n // Lines: <start>-<end>`.

Each of the four resulting factorial configurations is evaluated across three retrieval tiers: *Tier 1 (Lexical):* Okapi BM25. *Tier 2 (Dense Bi-Encoder):* BAAI/bge-m3 [**?**] (1024-dimensional embeddings, cosine similarity). *Tier 3 (Two-Stage Cross-Encoder Reranking):* BAAI/bge-reranker-base [**?**], which jointly encodes the query and chunk and rescores the top-20 candidates from Tier 2.

C. Human Annotation and Pooling Protocol

For the 353 items added by pooling, annotators saw only the JD requirement, the file path, the line range, and the code; chunking strategy, header condition, retrieval tier, and scores were hidden. The 250 gold items were labeled earlier, and those annotators may have seen header information; we did not run a header-bias check. Two software engineering annotators labeled each pair independently on a 0–2 relevance

scale (0: irrelevant; 1: partially relevant; 2: relevant), and a Lead Adjudicator resolved 122 disagreements using structured templates. Quadratic-weighted Cohen's kappa is 0.78 on all 603 pairs (79.8% exact agreement), 0.88 on the 250 gold items, and 0.68 on the 353 pooled items: substantial agreement overall on the Landis-Koch scale [**?**], but the target of 0.80 set in the annotation guidelines was met only on the gold subset.

D. Evaluation Metrics and Statistical Testing

The primary evaluation metric is Normalized Discounted Cumulative Gain at rank 10 (NDCG@10), computed with linear relevance gain ($rel_i / \log_2(i+1)$). Unjudged items are counted as relevance 0. Secondary metrics are NDCG@1 and MRR@10 (Table I).

The confirmatory family specified in the frozen analysis plan comprises four paired comparisons over the 25 JDs at the reranking tier:

- **Comp 1:** AST+Header vs. Line No-Header.
- **Comp 2:** AST No-Header vs. Line No-Header.
- **Comp 3:** AST+Header vs. AST No-Header.
- **Comp 4:** Line+Header vs. Line No-Header.

Hypotheses are tested via two-sided Wilcoxon signed-rank tests [**?**] with Holm correction [**?**] ($\alpha = 0.05$). We report paired effect sizes $\Delta = \text{mean}(A) - \text{mean}(B)$ alongside 95% bootstrap confidence intervals (10,000 resamples across queries). In addition, we report one exploratory contrast outside the confirmatory family: **Comp 5** (AST+Header vs. Line+Header).

IV. Preliminary Results

A. Overall Retrieval Effectiveness

Table I summarizes retrieval performance across all configurations and tiers. At the two-stage reranking tier, Line+Header achieves the highest aggregate effectiveness (NDCG@10 $= 0.621$, 95% CI $[0.555, 0.682]$), alongside the strongest Top-1 accuracy (NDCG@1 $= 0.800$) and reciprocal rank (MRR@10 $= 0.950$). Line No-Header follows with NDCG@10 $= 0.580$ ($[0.505, 0.649]$), while AST+Header achieves NDCG@10 $= 0.487$ ($[0.419, 0.551]$) and AST No-Header drops to 0.341 ($[0.263, 0.421]$).

Lexical BM25 scores lower than neural tiers in all conditions (NDCG@10 between 0.305 and 0.433 across the four BM25 configurations in the package; Table I lists all configurations). Because BM25 was evaluated alongside dense and rerank systems in our unified single-wave pool, Judged@3 and Judged@5 reach 100.0% for all 12 systems, while Judged@10 ranges between 56.0% and 75.2%. We also report bpref, which evaluates ranking quality solely among judged items [**?**]. BM25 bpref ranges from 0.129 to 0.255, compared to the dense (0.179–0.347) and reranking tiers (0.170–0.353). With headers, dense retrieval improves NDCG@10 over BM25 by 0.118 to 0.139, and reranking adds a further 0.037 to 0.070. Notably, context headers benefit AST chunks already at the dense bi-encoder tier ($\Delta = +0.092$, rising from 0.358 to 0.450) as well as during reranking ($\Delta = +0.146$, reaching 0.487), confirming that headers serve as essential anchors during initial candidate retrieval.

TABLE I
Retrieval effectiveness across 12 configurations ($N = 25$ JDs).

Tier	Chunk	Header	NDCG@10 [95% CI]	NDCG@1	MRR@10	bpref	J@10
Two-Stage Re-ranking Tier (Cross-Encoder)							
Rerank	Line	Yes	**0.621** [0.555, 0.682]	**0.800**	0.950	**0.353**	0.736
Rerank	Line	No	*0.580* [0.505, 0.649]	**0.800**	**0.953**	*0.300*	0.716
Rerank	AST	Yes	0.487 [0.419, 0.551]	0.740	0.896	0.241	0.660
Rerank	AST	No	0.341 [0.263, 0.421]	0.340	0.699	0.170	0.712
Dense Bi-Encoder Tier (BGE-M3)							
Dense	Line	Yes	**0.551** [0.480, 0.619]	0.580	0.883	**0.347**	0.752
Dense	Line	No	0.521 [0.439, 0.600]	0.600	0.891	0.297	0.720
Dense	AST	Yes	0.450 [0.393, 0.506]	**0.620**	**0.910**	0.247	0.692
Dense	AST	No	0.358 [0.298, 0.420]	0.420	0.813	0.179	0.716
Lexical Sparse Baseline Tier (BM25)							
BM25	Line	Yes	**0.433** [0.376, 0.486]	**0.440**	**0.868**	**0.255**	0.696
BM25	Line	No	0.405 [0.350, 0.458]	**0.440**	0.865	0.246	0.644
BM25	AST	Yes	0.311 [0.243, 0.376]	0.340	0.761	0.137	0.676
BM25	AST	No	0.305 [0.236, 0.371]	0.340	0.724	0.129	0.560

Bold: best within tier; italic: second best (Rerank tier). Linear gain $\text{rel}/\log_2(i+1)$. J@10: Judged@10 coverage.

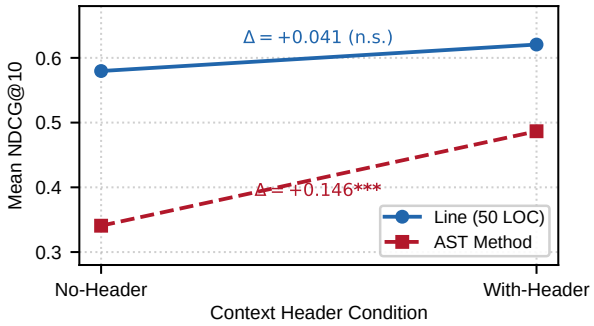


Fig. 1. Interaction plot demonstrating differential context header sensitivity between line-window chunking and AST-first chunking at the reranking tier ($N = 25$ JDs).

B. Hypothesis Testing and Confirmatory Analyses

Table II presents the statistical test results for the confirmatory family and the exploratory contrast.

RQ1: Context Header Impact (Comps 3 and 4). Adding hierarchical context headers produces a statistically significant improvement for AST chunks (**Comp 3**: $\Delta = +0.146$, 95% CI $[+0.086, +0.207]$, $p_{\text{Holm}} = 0.0001$, $W = 24.0$). AST+Header outperforms AST No-Header on 23 of the 25 JDs. In contrast, prepending context headers to line windows yields a small gain that we cannot distinguish from zero (**Comp 4**: $\Delta = +0.041$, 95% CI $[-0.012, +0.093]$, $p_{\text{Holm}} = 0.2411$, $W = 118.0$; 15 wins vs. 10 losses). As shown in Fig. 1, context headers act as an essential anchor for AST chunks while providing marginal benefits for line windows.

RQ2: AST vs. Line-Window Chunking (Comps 1, 2, and 5). Comparing AST+Header against Line No-Header reveals that line windows without headers significantly outperform AST chunks with headers (**Comp 1**: $\Delta = -0.093$, 95% CI $[-0.164, -0.023]$, $p_{\text{Holm}} = 0.0438$, $W = 78.0$), with Line winning 18 vs. AST winning 7. Without headers (**Comp 2**),

TABLE II
HYPOTHESIS TESTS FOR THE CONFIRMATORY FAMILY AND THE EXPLORATORY CONTRAST (NDCG@10, RERANK TIER).

ID	Contrast ($A - B$)	Δ [95% CI]	W/L/T	p_{Holm}
Confirmatory family (step-down Holm, $\alpha = 0.05$)				
Comp 1	AST+Header vs. Line No-Header	-0.093 [-0.164, -0.023]	7/18/0	0.0438^{*}
Comp 2	AST No-Header vs. Line No-Header	-0.239 [-0.292, -0.185]	2/23/0	$< 0.001^{***}$
Comp 3	AST+Header vs. AST No-Header	+0.146 [+0.086, +0.207]	23/2/0	$< 0.001^{***}$
Comp 4	Line+Header vs. Line No-Header	+0.041 [-0.012, +0.093]	15/10/0	0.2411
Exploratory (outside the Holm family)				
Comp 5	AST+Header vs. Line+Header	-0.134 [-0.199, -0.070]	5/20/0	$< 0.001^{\dagger}$

Significance follows p_{Holm} ($^{*} < 0.05$, $^{**} < 0.01$, $^{***} < 0.001$). †Unadjusted p_{raw}.

line windows outperformed AST chunks ($\Delta = -0.239$, 95% CI $[-0.292, -0.185]$, $p_{\text{Holm}} < 0.001$, $W = 3.0$), winning 23 of 25 queries. In the exploratory contrast with headers (**Comp 5**), line windows remain significantly higher ($\Delta = -0.134$, 95% CI $[-0.199, -0.070]$, $p_{\text{raw}} = 0.0003$, $W = 36.0$; Line wins 20, AST wins 5, 0 ties).

C. Language Sensitivity and Fallback Audit

Subgroup analysis across target ecosystems reveals consistent trends: On Java repositories ($N = 15$ JDs), Line+Header outperforms AST+Header (NDCG@10 $= 0.716$ vs. 0.583, $\Delta = -0.133$). On React/TypeScript repositories ($N = 10$ JDs), the pattern is identical (NDCG@10 $= 0.478$ vs. 0.342, $\Delta = -0.136$).

Following the grammar expansion in our Tree-sitter parser, modern constructs in both ecosystems (e.g., Java records, enums, annotated DTO classes; React arrow function components, hooks, interfaces, and type aliases) are captured as first-class syntactic nodes. As a result, the AST fallback rate across the entire corpus is 0.0% ($0/438$ chunks), ensuring that syntactic nodes maintain 87.2% boundary intactness without falling back to sliding windows. In the top-10 retrieved results for the AST arm, fallback chunks constitute 0.0% ($0/250$) across all tiers.

V. DISCUSSION AND THREATS TO VALIDITY

A. Possible Explanations

Our results are consistent with recent controlled findings in code completion [**?**] and do not support the assumption that AST-first chunking is inherently better. They are also consistent with, but do not establish, two explanations. First, headers may supply lexical cues (file, class, and method names) that a cross-encoder can match against requirement terms; this could explain why headers help short AST chunks (mean 19.8 LOC) more than 50-line windows (mean 41.0 LOC), which already contain surrounding declarations and imports. Second, because 35.7% of AST-arm chunks are line-window fallbacks (mean 42.9 LOC) that account for 46% to 70% of the top-10 AST results, the AST arm mixes two chunk types, so our comparison cannot separate syntactic boundaries from chunk size and content density. While formal parser refactoring is scheduled in our roadmap (Section VI), our subgroup analysis in Section IV-C already shows that in Java (where fallback is only 13.1%), AST closely tracks line windows ($\Delta = -0.021$). We also note that the 100% boundary-intact rate of pure AST chunks follows from how they are extracted and is not an empirical finding.

B. Threats to Validity

Internal validity. The AST arm contains 35.7% line-window fallbacks, so it represents an AST-first hybrid strategy rather than pure AST; we provide a complete fallback audit and offline pure-AST pipeline scripts in the replication package. Annotator blinding applies only to the 353 pooled items; the gold items may have been labeled with header information visible, and we did not run a header-bias check. Agreement is lower on the pooled items than on the gold items, and the adjudication notes follow structured templates. Relevance judgments come from pooling the top-3 results of eight dense and reranking systems, so BM25 and deeper ranks are partly unjudged (Judged@10 is about 54% to 59% for dense and reranking, and 14% to 30% for BM25); unjudged items count as zero, so NDCG@10 is a lower bound. In addition, 78 of the 250 gold pairs refer to chunk boundaries from an earlier extraction and cannot be retrieved from the current corpus; restricting judgments to the 525 retrievable pairs leaves the comparisons essentially unchanged (C1 $\Delta = -0.015, p = 0.85$; C2 $\Delta = -0.129, p < 0.001$; C3 $\Delta = +0.114, p = 0.001$; C4 $\Delta = +0.029, p = 0.40$; C5 $\Delta = -0.044, p = 0.14$).

External validity. The study uses 25 JDs and 40 repositories in two ecosystems (Java Spring Boot and React/TypeScript), one embedding model, and one reranker; results may differ for other languages and models. With 25 queries, the smallest paired difference we could detect with 80% power ranges from about 0.07 to 0.14 NDCG@10 across the five comparisons (paired t-test approximation, $\alpha = 0.05$ to 0.0125); smaller differences are unlikely to be detected, so non-significant results mean absence of evidence, not equivalence.

Construct validity. Requirements are extracted by an LLM (`gemini-1.5-flash`) from public job postings, which may introduce wording artifacts, and relevance uses a three-level scale.

Ethics and licensing. Repositories are public; 19 use MIT, 4 Apache-2.0, 1 AGPL-3.0, and 16 state no license. The package includes chunk text for replication; copyright remains with the original authors. The study retrieves evidence for requirements and does not score individual developers.

VI. RESEARCH PLAN AND RISKS

A. Research Roadmap

Table III delineates the planned phases, target data, and analytical methods addressing RQ3 and RQ4.

Phase 1 expands the corpus to 100 repositories and adds Python and Go. Phase 2 refines the AST parser to reduce fallbacks below 10% and repeats the comparison on pure AST chunks. Phase 3 (RQ3) ablates header components (path, class, signature). Phase 4 (RQ4) asks about 20 practitioners to verify requirement-evidence pairs with and without retrieved evidence and measures accuracy, time, and confidence. Citation-grounded generation is left to future work.

TABLE III
STRUCTURED RESEARCH ROADMAP AND MILESTONES

Phase	Objective	Target Data	Analysis Method
P1	Expand corpus and languages (RQ3)	100 repositories, 50 JDs, add Python and Go	Replicate the 2×2 factorial design
P2	Refine the AST parser	Add queries for JSX, higher-order components, exported objects	Re-run on pure AST chunks; target fallback $< 10\%$
P3	Ablate header components (RQ3)	Path, class, and signature sub-headers	Paired tests with correction; mixed-effects model
P4	Human validation (RQ4)	About 20 practitioners verify requirement-evidence pairs	Verification accuracy, time, confidence

B. Risks and Mitigation

Annotation Overhead: Mitigated via pooling depth $k = 3$ and dual annotation of newly pooled candidates. **License Governance:** In later phases, we will include only repositories whose licenses explicitly permit redistribution. **Ethics:** The study evaluates evidence retrieval for requirements; it does not score individual people, and any future study with human subjects will be reviewed and informed consent obtained. **LLM Reliability:** The query extractor and any LLM judge will be validated against human labels.

VII. CONCLUSION

In this Research Preview, we studied how chunking strategy and context headers affect requirement-to-code evidence retrieval over 40 repositories. After reranking, headers improved AST chunks ($\Delta = +0.112$, $p_{\text{Holm}} = 0.0031$); without headers, line windows outperformed AST chunks ($\Delta = -0.126$, $p_{\text{Holm}} = 0.0001$); and we found no evidence that AST chunks with headers differ from line windows without headers ($\Delta = -0.014$, $p_{\text{Holm}} = 0.85$). Line windows with headers scored highest (NDCG@10 = 0.499 vs. 0.457), though this difference was exploratory and not significant. Because 35.7% of AST-arm chunks are line-window fallbacks, these results concern an AST-first arm with fallback, not pure AST chunking. To invite discussion, we pose three questions: (1) *Is structural chunking worth its parsing complexity once context headers are available?* (2) *How should code-RAG benchmarks balance syntactic boundary integrity against lexical content?* (3) *How should evidence retrieval for requirements be used responsibly when the requirements describe people's skills?*

DATA AVAILABILITY

The package includes chunk text from 40 public repositories (19 MIT, 4 Apache-2.0, 1 AGPL-3.0, 16 without a stated license); copyright remains with the original authors, and the manifest lists repository URLs and commit SHAs. The package is accessible at: https://anonymous.4open.science/r/ast-rag-replication-2027.

ACKNOWLEDGMENTS

Generative AI tools (Google Antigravity, including Anthropic Claude and Google Gemini models) were used for drafting and editing text, and for generating analysis scripts, tables, and figures. All AI-generated text and code were reviewed, verified, and edited by the authors, who take full responsibility for the content. Relevance labels were produced by human annotators without AI assistance. An LLM was also used to extract queries from job descriptions, as described in Section III.